



# PROGRAMACIÓN ASÍNCRONA E INTEGRACION CON SQLITE

Este capítulo introduce la esencial programación asíncrona en Dart y su aplicación en Flutter, permitiendo la construcción de aplicaciones interactivas y eficientes. Se inicia con una explicación clara sobre Future, async y await en Dart, lo cual, en la práctica, habilita la integración de estas operaciones asíncronas en la interfaz de usuario en Flutter. Esto mejora la capacidad de respuesta de las aplicaciones mediante una gestión efectiva de estados y datos dinámicos.

A continuación, se guiará en la integración de bases de datos en proyectos Flutter, utilizando el paquete sqflite para brindar una experiencia rica y personalizada a los usuarios. Sqflite, siendo una implementación de SQLite para Flutter, permite almacenar, recuperar, modificar y eliminar datos de manera eficiente, directamente desde el dispositivo.

El ejercicio práctico de este capítulo guiará a través del proceso de integrar una base de datos en la aplicación de lista de planes que se ha estado desarrollando. Este paso no solo reforzará la comprensión de sqflite y las operaciones con bases de datos, sino que también mostrará cómo se puede mejorar significativamente la funcionalidad de las aplicaciones mediante la gestión de datos persistentes.

## 5.1 Dart asíncrono

Según Alessandria & Kayfitz, p. (2021, p. 248):

La programación asíncrona permite que tu aplicación complete tareas que consumen tiempo, como recuperar una imagen de la web o escribir algunos datos en un servidor web, mientras ejecuta otras tareas en paralelo y responde a la entrada del usuario. Esto mejora la experiencia del usuario y la calidad general de tu software.” [Traducción propia].

Lo anterior ayuda en el entendimiento de que la programación asíncrona permite que operaciones lentas se procesen en el "fondo" sin interrumpir la interacción del usuario. Esta capacidad de manejar múltiples tareas simultáneamente no solo mejora significativamente la percepción de la rapidez y fluidez de la aplicación por parte del usuario, sino que también optimiza el rendimiento general del software.

En esta sección, se estudiará inicialmente la clase *Future*, luego se explicarán algunas características como *async/await* y se cerrará la sección con el manejo de errores.

### 5.1.1 Explicación de Future

La clase *Future* constituye el pilar central de la programación asíncrona en Dart, para entender su funcionamiento, se puede establecer una analogía con el proceso de pedir una hamburguesa en un restaurante de comida rápida. Al realizar un pedido en el mostrador, al cliente se le entrega un recibo, este actúa como una promesa de que se le entregará una hamburguesa una vez esté preparada, sin necesidad de esperar en el mostrador hasta que esto ocurra, de manera similar, en la programación asíncrona, al solicitar una operación que tomará cierto tiempo en completarse, *Future* ofrece un "recibo" o promesa, este "recibo" asegura que el resultado estará disponible en el futuro, permitiendo que otras operaciones se ejecuten simultáneamente sin bloquear el flujo principal del programa.

Continuando con la analogía del restaurante, una vez que el cliente ha realizado su pedido y ha recogido su recibo, espera hasta que el personal llame su número para recoger su pedido. En este contexto, el recibo simboliza el *Future*. Dicho recibo actúa como un comprobante de la transacción, evidenciando que el cliente está aguardando la entrega de una hamburguesa. El recibo confirma la existencia de una promesa: la entrega de un valor específico, en este caso, la hamburguesa. Sin embargo, es importante destacar que la hamburguesa representa el valor futuro a recibir y no el *Future* en sí. Este último simplemente indica que el valor está siendo preparado y será entregado una vez completado, permitiendo así que el flujo de ejecución continúe sin interrupciones hasta que el resultado esté listo.

En el ámbito del desarrollo de software, un *Future* funciona como un marcador de posición para un valor que se espera recibir en el futuro. Esta característica resulta especialmente útil en operaciones como la recuperación de datos a través de la red. Por ejemplo, dentro del diseño de una interfaz de usuario (UI), es posible emplear un *Future<List<String>>* para representar una lista de elementos que eventualmente se mostrarán al usuario. Esta lista podría necesitar ser obtenida desde una fuente externa, como una API, mediante una

solicitud HTTP. En este contexto, el uso de *Future* permite comunicar a la interfaz de usuario que debe mostrar un indicador de carga mientras espera que el *Future* se resuelva, anticipando la llegada de una lista de cadenas de texto que posteriormente será exhibida. Este enfoque facilita la creación de interfaces de usuario dinámicas y responsivas, que pueden gestionar adecuadamente los tiempos de espera y la carga de datos asincrónica.

Los Futures permiten una gestión flexible de operaciones asincrónicas mediante su capacidad de ser "thenable". Esto significa que se puede aplicar el método `.then()` a un *Future* para definir acciones que se realizarán una vez que el *Future* se resuelva, es decir, cuando el valor esperado esté finalmente disponible. Utilizar `.then()` proporciona una manera estructurada de encadenar procesos subsiguientes, lo que permite un manejo eficiente y ordenado de las tareas dependientes de resultados asincrónicos. Este método es esencial en la programación asincrónica para asegurar que ciertas acciones solo se ejecuten tras la obtención de los datos necesarios, facilitando así la implementación de flujos de trabajo complejos que dependen de operaciones que pueden tardar en completarse, como las peticiones a APIs o la lectura de archivos. Un ejemplo de su sintaxis se puede observar a continuación:

```
myFutureMethod().then((returnValue) => ... do some code ... );
```

En el ejemplo, `myFutureMethod().then` toma un callback, que se ejecutará cuando el valor del futuro se resuelva, en el restaurante de hamburguesas, el callback es lo que decides hacer con la hamburguesa cuando la obtienes. El valor pasado al callback es el valor de retorno original del futuro.

En pseudocódigo, usar `.then()` se vería así: `orderBurger().then(eatBurger());`.

Para ilustrar el funcionamiento asincrónico de los Futures, se puede examinar el siguiente ejemplo, este demuestra que la ejecución de los Futures no sigue necesariamente el orden lineal en el que el código es escrito. Es decir, la presencia de un *Future* en el código no impide la ejecución de las instrucciones subsiguientes, esta característica subraya el principio de que un *Future* opera de manera no bloqueante, permitiendo que el flujo de ejecución del programa continúe mientras espera la resolución del *Future*, esta funcionalidad es esencial en la programación asincrónica, ya que facilita la realización de tareas dependientes de operaciones prolongadas, como peticiones de red o lecturas de archivos, sin detener la ejecución global del programa. Así, se promueve una gestión eficaz de los recursos y se mejora la experiencia de usuario al evitar retrasos innecesarios en la interfaz de usuario.

```
void main() {  
    print("A");  
    futurePrint(Duration(milliseconds: 1), "B")  
    .then((status) => print(status));  
    print("C");  
    futurePrint(Duration(milliseconds: 2), "D")
```

```
.then((status) => print(status));  
print("E");  
}  
Future<String> futurePrint(Duration dur, String msg) {  
  return Future.delayed(dur).then((onValue) => msg);  
}
```

Este ejemplo inicialmente imprime "A" de forma sincrónica, seguidamente, realiza una llamada a *futurePrint* con un retraso de 1 milisegundo para imprimir "B", pero continúa su ejecución sin esperar, imprimiendo "C" de manera inmediata. Posteriormente, se invoca nuevamente *futurePrint* con un retraso de 2 milisegundos para imprimir "D", prosiguiendo sin pausa para imprimir "E". Las funciones *futurePrint* se resuelven asincrónicamente después de sus respectivos retrasos, primero retornando "B" y luego "D", lo que demuestra el manejo asincrónico de Dart mediante Futures, donde el orden de ejecución del código no necesariamente coincide con el orden en el que se completan y se muestran los resultados.

En muchos lenguajes, incluyendo Dart, existen palabras clave especiales que hacen que la escritura de código asíncrono sea más concisa y legible, ee explorarán esas palabras clave a continuación.

### 5.1.2 Las palabras reservadas "async/await"

Las palabras clave *async* y *await* simplifican considerablemente la comprensión y el manejo de la programación asíncrona. Al declarar una función con *async*, se habilita la posibilidad de pausar la ejecución de dicha función en puntos específicos mediante *await*, hasta que se completen ciertas operaciones asíncronas. Esta pausa se efectúa en cada punto donde se encuentra *await*, lo que permite que la función espere la resolución de un proceso asíncrono antes de proceder. De esta manera, *async* y *await* facilitan la escritura de código que, a pesar de operar de manera asíncrona, se estructura de una forma que es conceptualmente similar al código sincrónico, permitiendo una secuencia lógica y clara de operaciones que mejora la legibilidad y mantenibilidad del código.

En ciertos escenarios, la correcta ejecución de procesos subsiguientes depende de la finalización de operaciones asíncronas previas. Por ejemplo, un proceso que necesita datos suministrados por una API externa no puede proceder hasta que se hayan obtenido y procesado dichos datos completamente. Al introducir las palabras clave *await* en este contexto, se instruye al código para que pause su ejecución en puntos específicos hasta que las operaciones asíncronas señaladas se hayan completado satisfactoriamente. Este mecanismo permite asegurar que las dependencias críticas para la ejecución de ciertas tareas sean resueltas antes de avanzar, garantizando así la integridad y el correcto flujo lógico del proceso.

```
void main() async {  
  print("A");  
  await futurePrint(Duration(milliseconds: 1), "B")  
}
```

```
        .then((status) => print(status));
    print("C");
    await futurePrint(Duration(milliseconds: 2), "D")
        .then((status) => print(status));
    print("E");
}

Future<String> futurePrint(Duration dur, String msg) {
    return Future.delayed(dur).then((onValue) => msg);
}
```

La esencia de este enfoque radica en la capacidad de transformar la ejecución de código asíncrono en una secuencia que aparenta ser síncrona, aunque esta idea puede parecer contraintuitiva inicialmente, su aplicación resulta ser de gran utilidad en determinados contextos. Por otro lado, es importante señalar que el ejemplo proporcionado anteriormente ha sido diseñado de forma intencionalmente simplificada para ilustrar el concepto. En la práctica, el mismo resultado podría de la siguiente manera:

```
void main() {
    print("A");
    print("B");
    print("C");
    print("D");
    print("E");
}
```

Esta técnica resulta efectiva debido a que, en el ejemplo proporcionado, se conoce de antemano la función específica del código asíncrono: imprimir letras. En escenarios reales, es común que se espere la recepción de información proveniente de una llamada HTTP y se desee "pausar" la ejecución del código hasta que dicha llamada se haya completado satisfactoriamente. El siguiente listado proporciona un pseudocódigo para la obtención de datos de manera asíncrona, ilustrando cómo se puede aplicar esta metodología para gestionar operaciones que dependen del resultado de solicitudes asíncronas.

```
void main() async {
    var user = await http.get("http://my-database.com/user/1");
    print(user);
}
```

En ocasiones, puede ocurrir que un *Future* no logre completar la tarea asignada; en estos casos, es decisivo implementar mecanismos de captura de errores, esto permite proceder de manera alternativa, asegurando que el programa pueda manejar la situación de forma adecuada y continuar su ejecución o informar del problema de manera controlada. La captura y gestión de errores en operaciones asíncronas son fundamentales para el desarrollo de aplicaciones resilientes y confiables. Es posible capturar errores en código asíncrono mediante

dos métodos, lo cual permite gestionar eficazmente tanto errores como llamadas de red fallidas, sin comprometer la experiencia del usuario.

### 5.1.3 Captura de errores con `CatchError`

La captura de errores en operaciones asíncronas se puede realizar utilizando el método `catchError` asociado a los `Futures`, este enfoque permite manejar excepciones y errores que surgen durante la ejecución de tareas asíncronas, como solicitudes de red o consultas a bases de datos. Al emplear `catchError`, se añade un mecanismo de seguridad que intercepta errores, proporcionando la oportunidad de ejecutar un bloque de código alternativo para gestionar la situación de error, de esta manera, se puede evitar que la aplicación falle inesperadamente y se mejora la experiencia de usuario, permitiendo que la aplicación responda de manera elegante ante fallos, ofreciendo mensajes de error claros o implementando lógicas de recuperación, asegurando así la continuidad y estabilidad de la aplicación.

La sintaxis necesaria se puede observar en el siguiente ejemplo:

```
Future futurePrint(Duration dur, String msg) async {  
  return Future.delayed(dur).then((onValue) => msg);  
}  
main() {  
  futurePrint(Duration(milliseconds: 2), "D")  
    .then((status) => print(status))  
    .catchError((err) => print(err));  
}
```

Además de cómo `async` y `await` simplifican la escritura de código asíncrono en comparación con el uso directo de objetos `Future`, existe un método más refinado para la captura de errores: el uso de bloques `try/catch`, una construcción presente en numerosos lenguajes de programación y Dart no es la excepción. Esta alternativa será examinada a continuación.

### 5.1.4 Captura de errores con `try` y `catch`

Los bloques `try/catch` resultan particularmente efectivos en el manejo de operaciones que emplean `async/await`. En la estructura de estos bloques, el código dentro de `try` se ejecuta independientemente de las circunstancias. En caso de que este se complete exitosamente, es decir, sin generar errores, el compilador ignorará el bloque `catch` y proseguirá con la ejecución del código subsiguiente como estaba previsto. Por otro lado, el bloque `catch` actúa como un mecanismo de protección, entrando en acción únicamente si se detecta un error dentro del bloque `try`, proporcionando así un medio para gestionar y responder ante posibles fallos en la ejecución del código.

```
Future futurePrint(Duration dur, String msg) async {  
  return Future.delayed(dur).then((onValue) => msg);  
}
```

```
}  
void main() async {  
  try {  
    print("A");  
    await futurePrint(Duration(milliseconds: 1), "B")  
      .then((status) => print(status));  
    print("C");  
    await futurePrint(Duration(milliseconds: 2), "D")  
      .then((status) => print(status))  
      .catchError((err) => print());  
    print("E");  
  } catch (err) {  
    print("Err!! -- $err");  
  }  
}
```

La programación asincrónica potencia la reactividad y eficiencia de las aplicaciones permitiendo que operaciones complejas, tales como la recuperación de datos, se ejecuten de manera secundaria, sin interrumpir la experiencia del usuario. Esta metodología armoniza perfectamente con el manejo de bases de datos, cuya implementación y estudio naturalmente se convierten en el siguiente paso crítico para el desarrollo de aplicaciones robustas y dinámicas.

## 5.2 Bases de datos

Las bases de datos constituyen el eje central sobre el que giran la mayoría de las aplicaciones modernas, proporcionando un sistema estructurado para almacenar, recuperar y gestionar datos de manera eficiente. Son fundamentales para el desarrollo de soluciones tecnológicas, desde sitios web hasta aplicaciones móviles y sistemas empresariales. Dentro de este amplio espectro, las bases de datos relacionales se destacan por su capacidad para estructurar la información en tablas relacionadas entre sí, lo que facilita consultas complejas y la integridad de los datos a través de relaciones definidas, como las claves primarias y foráneas.

En el ámbito del desarrollo móvil, según Alessandria & Safari, p. (2020, p. 188) “SQLite es una muy buena opción para la persistencia de datos en Android e iOS porque es fácil de implementar y es seguro, de dominio público, multiplataforma y compacto.” [Traducción propia].

En ese sentido, SQLite es una implementación de base de datos relacional ligera que sobresale por su simplicidad y eficiencia, sin requerir una configuración de servidor dedicado, lo que la hace ideal para aplicaciones móviles que necesitan una base de datos compacta y autónoma. Su integración en el entorno de desarrollo de Flutter permite manejar datos con facilidad, aprovechando SQL como lenguaje de consulta para realizar operaciones de datos. La combinación de SQLite con Flutter facilita la creación de aplicaciones móviles con capacidades de gestión de datos robustas, permitiendo almacenar, consultar y manipular datos directamente desde el dispositivo, lo que contribuye a una experiencia de usuario fluida y una gestión eficiente de la información. Los componentes principales de una base de datos SQLite incluyen:

- **Tablas:** Son el núcleo de una base de datos SQLite, donde se almacenan los datos, cada tabla está organizada en filas y columnas, con cada fila representando un registro único y cada columna un campo específico de información. Las tablas permiten estructurar los datos de forma lógica y accesible.
- **Claves Primarias y Foráneas:** Son fundamentales para establecer relaciones entre tablas, la clave primaria es un identificador único para los registros dentro de una tabla, mientras que las claves foráneas vinculan registros entre diferentes tablas, manteniendo la integridad referencial y permitiendo consultas relacionales.
- **Índices:** Facilitan la búsqueda rápida de registros dentro de las tablas, mejorando significativamente el rendimiento de las consultas. Un índice en SQLite se crea sobre una o más columnas de una tabla, lo que permite una recuperación de datos más eficiente.
- **Vistas:** Son resultados de consultas almacenados que se pueden utilizar como si fueran tablas, las vistas permiten simplificar consultas complejas, agrupando operaciones en un solo objeto virtual que puede ser reutilizado, mejorando así la legibilidad y mantenimiento del código.
- **Triggers (Desencadenadores):** Son procedimientos que se ejecutan automáticamente en respuesta a eventos específicos en las tablas, como inserciones, actualizaciones o eliminaciones. Los triggers permiten automatizar la gestión de la integridad de los datos y realizar operaciones de mantenimiento o auditoría de manera transparente.
- **Transacciones:** Permiten agrupar múltiples operaciones en una única unidad de trabajo que se ejecuta de manera atómica. Las transacciones aseguran la consistencia de los datos, permitiendo que múltiples cambios se confirmen o se reviertan juntos.

Estos componentes trabajan conjuntamente para ofrecer un sistema de gestión de datos robusto, eficiente y altamente adaptable, haciendo de SQLite una opción popular para el desarrollo de aplicaciones que requieren una solución de base de datos confiable y de fácil integración.

Además, SQLite en Flutter utiliza SQL (Structured Query Language: Lenguaje de Consultas Estructurado) como lenguaje de consulta para interactuar con la base de datos, comprender los principios de SQL permitirá realizar consultas y operaciones de manipulación de datos de manera efectiva. A continuación, se presenta una exploración detallada de las operaciones fundamentales que permiten interactuar con los datos almacenados en bases de datos relacionales:

### 5.2.1 Selección

Una de las tareas fundamentales en SQL consiste en la formulación de consultas, proceso que se realiza mediante el comando SELECT, este comando se emplea para especificar las columnas que se desean obtener como resultado de la base de datos, seguido del nombre de la tabla de donde se extraerán los datos. Por ejemplo:

```
// 1
SELECT name, address FROM Customers;
// 2
SELECT * FROM Customers;
```



```
// 3  
SELECT name, address FROM Customers WHERE name LIKE 'A%';
```

A continuación, se describe lo que sucede en el código anterior:

1. Devuelve las columnas de nombre y dirección de la tabla Customers.
2. Utilizando \*, devuelve todas las columnas de la tabla especificada.
3. Utiliza WHERE para filtrar los datos devueltos, en este caso, solo devuelve datos donde “name” comienza con A.

## 5.2.2 Agregación

La operación de inserción, ejecutada a través del comando INSERT en SQL, es esencial para agregar nuevos registros a una tabla dentro de una base de datos relacional, esta acción permite especificar y asignar valores a los diferentes campos de un registro conforme a la estructura y restricciones definidas para la tabla. Mediante el uso de INSERT, los usuarios pueden enriquecer la base de datos con información actualizada, manteniendo la relevancia y utilidad del conjunto de datos. La correcta implementación de este comando es importante para asegurar la integridad de los datos, ya que requiere que los valores insertados cumplan con las reglas de tipo de datos, restricciones de clave única y relaciones de integridad referencial establecidas en el esquema de la base de datos. Por ejemplo, es posible agregar datos utilizando la declaración:

```
INSERT INTO Customers (NAME, ADDRESS) VALUES (value1, value2);
```

Si bien no es necesario enumerar todas las columnas, si deseas agregar todos los valores, los valores deben estar en el mismo orden en que se definieron las columnas, es una buena práctica enumerar los nombres de las columnas siempre que insertes datos.

## 5.2.3 Eliminación

La operación de eliminación en SQL, realizada mediante el comando DELETE, juega un papel significativo en la gestión de datos dentro de bases de datos relacionales. Este comando permite remover uno o más registros de una tabla, basándose en criterios específicos definidos por el usuario. La eliminación se debe manejar con precaución, dado su impacto directo sobre la integridad de los datos y la posible afectación a las relaciones entre tablas. La habilidad para especificar condiciones precisas para la eliminación asegura que solo los registros que cumplan con dichos criterios sean eliminados, evitando así la pérdida accidental de información valiosa. Esta operación es fundamental para mantener la relevancia y actualización del conjunto de datos, permitiendo la gestión eficiente del almacenamiento y la coherencia de la información dentro de la base de datos. Para eliminar datos, es posible utilizar la declaración DELETE de la siguiente manera:

```
DELETE FROM Customers WHERE id = '1';
```

Si no se utiliza la cláusula WHERE, se eliminarán todos los datos de la tabla. En este contexto, se procede a eliminar el registro del cliente cuyo “id” es igual a 1. Es posible aplicar condiciones más generales para la

eliminación. Por ejemplo, se pueden eliminar todos los registros de clientes que pertenezcan a una ciudad específica, demostrando la flexibilidad y potencia de las operaciones de eliminación condicional en la gestión de bases de datos.

### 5.2.4 Actualización

La operación de actualización, ejecutada mediante el comando `UPDATE` en SQL, es fundamental para modificar los valores de uno o más campos en los registros existentes de una tabla, conforme a criterios específicos establecidos por la cláusula `WHERE`. Esta capacidad de alterar datos sin necesidad de remover y reinsertar registros facilita la gestión eficaz y precisa de la información almacenada, permitiendo ajustes como la actualización de direcciones de clientes, cambios en precios de productos o la corrección de errores tipográficos. La ejecución cuidadosa de `UPDATE` asegura la actualización coherente de los datos, manteniendo la integridad y la relevancia del conjunto de datos a lo largo del tiempo. Al igual que con otras operaciones críticas, el uso prudente de la cláusula `WHERE` es esencial para evitar actualizaciones no intencionadas que puedan afectar la precisión de la base de datos. Se utiliza `UPDATE` para actualizar tus datos con la siguiente sintaxis:

```
UPDATE customers
SET phone = '555-12345',
WHERE id = '1';
```

Esto actualiza el número de teléfono del cliente cuyo “id” es igual a 1.

Para lograr una integración efectiva de SQLite en aplicaciones Flutter, es esencial contar con herramientas especializadas que simplifiquen este proceso. A continuación, se analizará un paquete que facilita esta tarea, proporcionando soluciones prácticas y eficientes para la gestión de bases de datos SQLite en entornos de desarrollo Flutter.

## 5.3 El paquete sqflite de Flutter

El paquete “sqflite” emerge como una herramienta fundamental para la integración de bases de datos SQLite en aplicaciones Flutter, “sqflite” ofrece una interfaz sencilla y robusta que permite aprovechar las capacidades de SQLite para el almacenamiento local de datos en aplicaciones móviles. Con su amplia gama de funcionalidades, que van desde la creación y gestión de bases de datos hasta la ejecución de consultas avanzadas, “sqflite” proporciona una solución completa y versátil para las necesidades de gestión de datos en proyectos Flutter. En esta guía, se explora en detalle cómo utilizar “sqflite” para crear y mantener bases de datos locales, ejecutar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) y realizar consultas complejas, todo ello con el objetivo de potenciar el desarrollo de aplicaciones móviles de alto rendimiento y funcionalidad.

### 5.3.1 Integración con Flutter

La integración de “sqlite” con Flutter conlleva diversos pasos y consideraciones clave para habilitar el almacenamiento y gestión de datos locales en las aplicaciones Flutter mediante SQLite. A continuación, se describen detalladamente el proceso y los aspectos a tener en cuenta:

El primer paso para integrar “sqlite” en un proyecto Flutter es incluir la dependencia en tu archivo “pubspec.yaml”, esto se hace añadiendo la línea correspondiente a “sqlite” en la sección de dependencias:

```
dependencies:  
  flutter:  
    sdk: flutter  
  sqflite: ^[última_version]
```

Es importante verificar la versión de “sqlite” en “pub.dev” para asegurarse de que se está utilizando la versión más actual. Una vez añadida la dependencia y actualizado el proyecto (usualmente a través de un IDE o ejecutando “flutter pub get” en la terminal), es necesario importar el paquete “sqlite” en los archivos Dart donde se utilizará, esto se hace de la siguiente manera:

```
import 'package:sqflite/sqflite.dart';
```

Ahora procedamos a revisar cómo se realiza la apertura de la base de datos.

### 5.3.2 Apertura de la base de datos

Es importante recordar que una base de datos SQLite se almacena como un archivo, por lo que es necesario incluir un paquete adicional que permita la apertura de dicho archivo, ese paquete se denomina “path”, agregue la dependencia de la siguiente forma:

```
dependencies:  
  flutter:  
    sdk: flutter  
  sqflite: ^[última_version]  
  path: ^[última_version]
```

De igual manera, es necesario importar el paquete donde se utilizará:

```
import 'package:path/path.dart';
```

Ahora, se deberá utilizar la función *openDatabase* de *sqflite* para abrir la base de datos, es posible especificar la versión de la base de datos y proporcionar funciones de callback como *onCreate*, *onUpgrade* y *onDowngrade*, que se utilizarán para inicializar la base de datos, manejar actualizaciones o retrocesos de versión, respectivamente.

```
Future<Database> _initDB() async {  
  final dbPath = await getDatabasesPath();  
  final path = join(dbPath, 'my_database.db');  
  return await openDatabase(  
    path,  
    version: 1,  
    onCreate: _createDB,  
  );  
}
```

Este proceso inicializa y abre una base de datos SQLite en el dispositivo, lista para ser utilizada para almacenar, recuperar, actualizar o eliminar datos según sea necesario. Es necesario recordar que todas las operaciones de base de datos deben ser realizadas de manera asíncrona (con *async* y *await*) para no bloquear el hilo principal de la aplicación.

### 5.3.3 Creación de tablas

Para crear tablas en “sqlite”, se utilizan sentencias SQL dentro del método *onCreate*, en este método, se definen las estructuras de las tablas con sus respectivos campos y tipos de datos. Además, se pueden establecer restricciones como claves primarias, claves foráneas y restricciones de unicidad. Una vez definidas las sentencias de creación de las tablas, se ejecutan utilizando el método *execute* proporcionado por el objeto *Database*, esto garantiza que las tablas se creen correctamente en la base de datos SQLite y estén listas para ser utilizadas para almacenar y recuperar datos en la aplicación Flutter.

A continuación, se describen los pasos necesarios para crear una tabla de manera más detallada.

#### Definir las sentencias SQL para crear tablas

Primero, se necesita definir las sentencias SQL que se utilizarán para crear las tablas en su base de datos, estas sentencias deben especificar el nombre de la tabla, las columnas, los tipos de datos de cada columna y cualquier clave primaria o restricción que se desee aplicar.

```
CREATE TABLE Users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT NOT NULL,  
  email TEXT NOT NULL UNIQUE,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

#### Implementar el onCreate

La función *onCreate* se llama cuando la base de datos se crea por primera vez, aquí es donde se ejecuta las sentencias SQL para crear las tablas.

```
Future _createDB(Database db, int version) async {  
    await db.execute('CREATE TABLE Users (id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT NOT NULL, email TEXT NOT NULL UNIQUE, created_at TIMESTAMP NOT NULL  
    DEFAULT CURRENT_TIMESTAMP)');  
}
```

### Implementar el onUpgrade

La función *onUpgrade* se invoca cuando la versión de la base de datos se incrementa, se utiliza para modificar el esquema de la base de datos, como agregar nuevas tablas o modificar tablas existentes.

### Implementar onDowngrade

La función *onDowngrade* es menos común y se utiliza para manejar los casos en los que la versión de la base de datos necesita ser revertida a una versión anterior, su implementación puede variar según las necesidades específicas de la aplicación y la complejidad del esquema de la base de datos.

## 5.3.4 Operaciones CRUD

Es posible realizar operaciones básicas de crear, leer, actualizar y eliminar (CRUD) sobre los datos almacenados, utilizando comandos SQL o los métodos “helper” proporcionados por SQLite.

### Crear (Insertar)

Para insertar una nueva tupla en una tabla, se dispone del método *insert* ofrecido por “sqlite”, el cual constituye una función auxiliar diseñada para facilitar la construcción e inserción de datos de manera sencilla. Otra alternativa es escribir directamente una sentencia SQL de inserción. Cuando se opta por el método *insert*, se sigue un procedimiento más conveniente y estructurado, lo que permite agregar registros a la tabla de forma eficiente y sin necesidad de escribir complejas sentencias SQL manualmente.

La sintaxis usando el método *insert* es la siguiente:

```
Future<void> insertUser(Database db, Map<String, dynamic> user) async {  
    await db.insert(  
        'Users',  
        user,  
        conflictAlgorithm: ConflictAlgorithm.replace,  
    );  
}  
// Uso:  
final user = {  
    'username': 'john_doe',  
    'email': 'john.doe@example.com',  
};  
await insertUser(db, user);
```

La alternativa usando SQL se puede observar a continuación:

```
Future<void> insertUserSql(Database db, String username, String email) async {  
  await db.execute(  
    'INSERT INTO Users (username, email) VALUES (?, ?)',  
    [username, email],  
  );  
}  
// Uso:  
await insertUserSql(db, 'john_doe', 'john.doe@example.com');
```

### Leer (Consultar)

Para realizar consultas SELECT utilizando “sqlite” en Flutter, se emplea el método *query* proporcionado por esta librería, que facilita la ejecución de consultas SQL de manera eficiente y estructurada. Al utilizar este método, se pueden especificar los criterios de selección, como las columnas a recuperar, las condiciones de filtrado, el ordenamiento de los resultados y la agrupación de datos, todo ello de acuerdo con la sintaxis estándar de SQL. Además, “sqlite” ofrece la opción de utilizar “placeholders” en las consultas para evitar la inserción directa de valores y prevenir posibles vulnerabilidades de seguridad. En resumen, el uso del método *query* de “sqlite” proporciona una manera segura y flexible de recuperar datos de la base de datos SQLite en aplicaciones Flutter, permitiendo una manipulación precisa y eficiente de la información almacenada.

```
Future<List<Map<String, dynamic>>> getUsers(Database db) async {  
  final List<Map<String, dynamic>> users = await db.query('Users');  
  return users;  
}  
// Uso:  
final users = await getUsers(db);
```

La alternativa usando SQL tiene la siguiente sintaxis:

```
Future<List<Map<String, dynamic>>> getUsersSql(Database db) async {  
  final List<Map<String, dynamic>> users = await db.rawQuery('SELECT * FROM Users');  
  return users;  
}  
// Uso:  
final users = await getUsersSql(db);
```

### Actualizar

Para actualizar registros en una base de datos SQLite utilizando “sqlite”, se hace uso del método *update* proporcionado por esta biblioteca, este método permite modificar los valores de uno o más campos en los registros existentes de una tabla, según los criterios especificados en la sentencia SQL de actualización. Al

utilizar el método *update*, se pueden establecer condiciones de filtrado para seleccionar los registros que se van a actualizar, así como definir los nuevos valores que se asignarán a los campos correspondientes. Además, “sqflite” ofrece la posibilidad de utilizar “placeholders” en las consultas de actualización para evitar la inserción directa de valores y prevenir posibles vulnerabilidades de seguridad. En resumen, el método *update* de “sqflite” proporciona una manera segura y eficiente de modificar datos en la base de datos SQLite en aplicaciones Flutter, permitiendo una manipulación precisa y controlada de la información almacenada. Para actualizar una tupla existente en una tabla, se puede usar el método *update* de la siguiente manera:

```
Future<void> updateUser(Database db, int id, Map<String, dynamic> user) async {
  await db.update(
    'Users',
    user,
    where: 'id = ?',
    whereArgs: [id],
  );
}
// Uso:
final user = {
  'username': 'johnny_doe',
  'email': 'johnny.doe@example.com',
};
await updateUser(db, 1, user); // Asumiendo que el ID del usuario a actualizar es 1
```

La sintaxis usando SQL directamente es la siguiente:

```
Future<void> updateUserSql(Database db, int id, String username, String email) async {
  await db.rawUpdate(
    'UPDATE Users SET username = ?, email = ? WHERE id = ?',
    [username, email, id],
  );
}
// Uso:
await updateUserSql(db, 1, 'johnny_doe', 'johnny.doe@example.com');
```

## Eliminar

Para eliminar registros de una base de datos SQLite utilizando “sqflite”, se emplea el método *delete* proporcionado por esta librería. Este método permite eliminar uno o más registros de una tabla, según los criterios especificados en la sentencia SQL de eliminación. Al utilizar el método *delete*, se pueden establecer condiciones de filtrado para seleccionar los registros que se van a eliminar, lo que ofrece flexibilidad en la gestión de datos. Además, “sqflite” ofrece la opción de utilizar “placeholders” en las consultas de eliminación para evitar la inserción directa de valores y prevenir posibles vulnerabilidades de seguridad. En resumen, el

método *delete* de “sqlite” proporciona una manera segura y eficiente de eliminar datos de la base de datos SQLite en aplicaciones Flutter, permitiendo una gestión precisa y controlada de la información almacenada. Para eliminar una tupla, se puede utilizar el método *delete* de la siguiente manera:

```
Future<void> deleteUser(Database db, int id) async {  
  await db.delete(  
    'Users',  
    where: 'id = ?',  
    whereArgs: [id],  
  );  
}  
// Uso:  
await deleteUser(db, 1); // Asumiendo que el ID del usuario a eliminar es 1
```

La alternativa usando SQL, es la siguiente:

```
Future<void> deleteUserSql(Database db, int id) async {  
  await db.rawDelete('DELETE FROM Users WHERE id = ?', [id]);  
}  
// Uso:  
await deleteUserSql(db, 1);
```

En resumen, es posible hacer operaciones CRUD utilizando métodos “helper” o comandos SQL cuando se emplea el paquete “sqlite”.

## 5.4 Ejercicio: manejo de memoria persistente en la aplicación de lista de planes

En el presente ejercicio, se procederá a incorporar una base de datos SQLite a la aplicación que fue desarrollada en el Capítulo 4, el enfoque principal será en la implementación de las funcionalidades para añadir y eliminar planes en una tabla de base de datos. Para llevar a cabo estas operaciones de manera efectiva, es esencial asegurar inicialmente la correcta organización y estructuración de los archivos y carpetas requeridos, esta estructura se puede observar en la siguiente imagen.



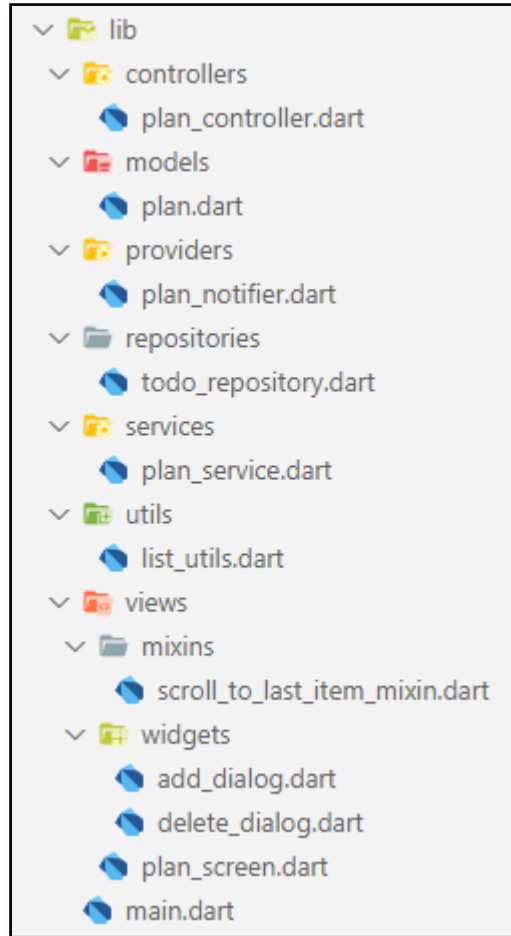


Figura V.1. Estructura del directorio lib de la aplicación

Se observará que la innovación radica en la creación de las carpetas denominadas "repositories" y "services". Asimismo, se destacan los archivos: "todo\_repository.dart" y "plan\_service.dart".

### 5.4.1 Instrucciones

Inicialmente, se enfocará en el modelo Plan, el cual deberá adaptarse para gestionar tanto la creación de un plan a partir de un mapa, como también, la exportación de un plan en formato de mapa. Posteriormente, se abordará el desarrollo de *TodoRepository*, la clase responsable de gestionar la base de datos SQLite. Seguidamente, se procederá a abstraer la funcionalidad correspondiente en *PlanService*, lo que implicará realizar ajustes tanto en el controlador como en los proveedores, estos cambios demandarán también adaptaciones relacionadas con la consistencia en la programación asíncrona.

Al concluir el ejercicio, el resultado en términos de diseño no mostrará diferencias significativas respecto al presentado en el Capítulo 4, la principal distinción se encuentra en la inclusión de memoria persistente a través del manejo de una base de datos SQLite. Por consiguiente, las vistas serán las siguientes:

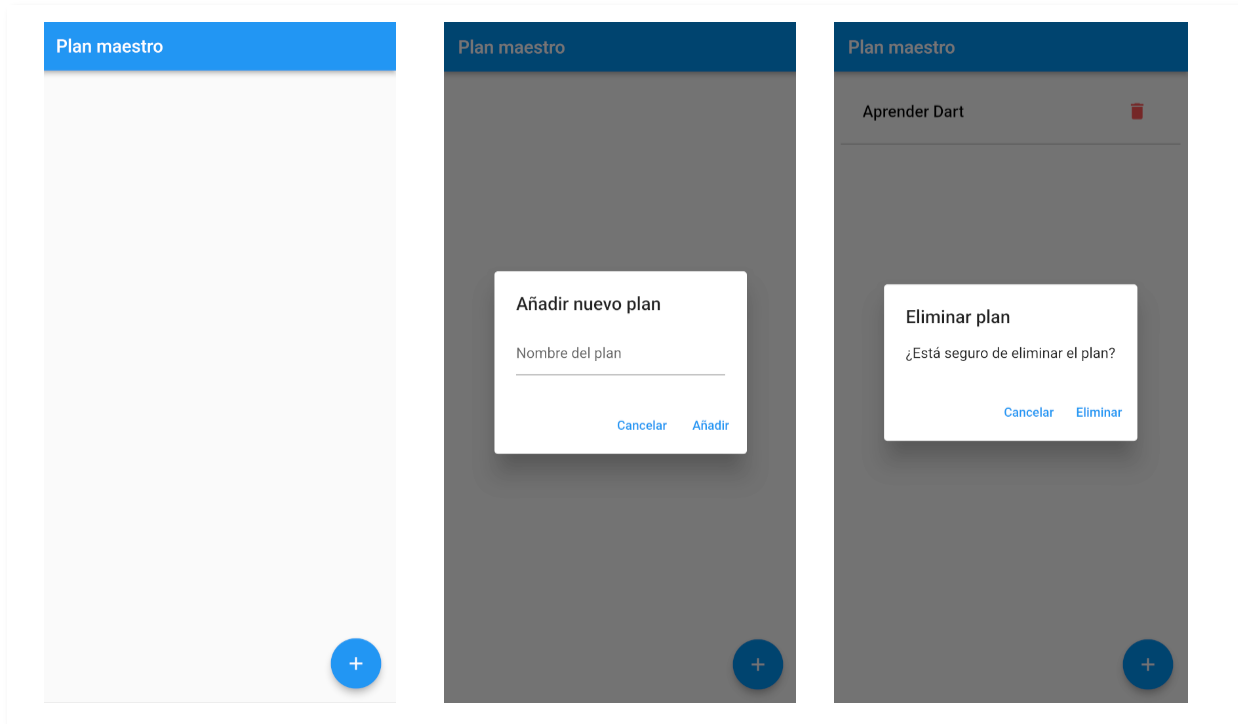


Figura V.2. Pantallas de la APP lista de planes

### Requisitos Previos.

Para iniciar el ejercicio, es necesario asegurarse de haber completado el ejercicio de la Sección 4 del Capítulo 4 y disponer de la aplicación resultante.

## 5.4.2 Modificación del modelo Plan

Inicialmente, se enfocará el trabajo en el desarrollo del modelo *Plan*, al cual se le incorporarán un par de funcionalidades, entre ellas, se destacan la implementación de un constructor que permita la creación de instancias a partir de un mapa y una función encargada de convertir el objeto Plan en un mapa. Adicionalmente, se añadirá una propiedad para manejar un identificador único para cada Plan.

models/plan.dart

```
class Plan {  
  final int? id;  
  final String name;  
  Plan({this.id, this.name = ''});  
  factory Plan.fromMap(Map<String, dynamic> map) {  
    return Plan(  
      id: map['id'],  
      name: map['name'],  
    );  
  }  
}
```

```

    }
    Map<String, dynamic> toMap() {
      return {
        if (id != null) 'id': id,
        'name': name,
      };
    }
  }
}

```

El código define una clase *Plan*, utilizada para modelar un plan con dos propiedades: *id* (un entero opcional que puede ser nulo, lo que indica que no es obligatorio proporcionar un identificador al crear una instancia de *Plan*) y *name* (una cadena que por defecto es una cadena vacía si no se proporciona un nombre al instanciar *Plan*).

El constructor nombrado *fromMap* permite crear una instancia de *Plan* a partir de un mapa *Map<String, dynamic>* que representa los datos de un plan, facilitando la deserialización de datos, por ejemplo, provenientes de una base de datos o de una respuesta de API.

El método *toMap* convierte la instancia de *Plan* en un mapa, útil para serializar el objeto para operaciones como almacenamiento en base de datos o envío de datos a través de una API, incluyendo condicionalmente el *id* solo si este no es nulo, asegurando que solo se incluyan datos válidos o necesarios en la representación del mapa.

### 5.4.3 Implementación del repositorio

El repositorio actúa como la clase fundamental que gestiona la interacción directa con la base de datos, sirviendo de puente entre esta y la lógica de la aplicación. Para asegurar una integración eficaz con SQLite, resulta esencial añadir los paquetes “sqlite”, que facilita las operaciones de la base de datos y “path”, que proporciona herramientas para definir las ubicaciones de los archivos de la base de datos.

repositories/todo\_repository.dart

```

import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
import '../models/plan.dart';
class TodoRepository {
  Database? _database;
  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB();
    return _database!;
  }
  Future<Database> _initDB() async {

```

```

    final dbPath = await getDatabasesPath();
    final path = join(dbPath, 'plan_task.db');
    return await openDatabase(
        path,
        version: 1,
        onCreate: _createDB,
    );
}
Future _createDB(Database db, int version) async {
    await db.execute('CREATE TABLE Plans(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT)');
}
Future<List<Plan>> getAllPlans() async {
    final db = await database;
    final List<Map<String, dynamic>> planMaps = await db.query('Plans');
    return planMaps.map((planMap) => Plan.fromMap(planMap)).toList();
}
Future<int> insertPlan(Plan plan) async {
    final db = await database;
    int id = await db.insert('Plans', plan.toMap());
    return id;
}
Future<void> deletePlan(Plan plan) async {
    final db = await database;
    await db.delete(
        'Plans',
        where: 'id = ?',
        whereArgs: [plan.id],
    );
}
}

```

El código presentado define una clase *TodoRepository*, que actúa como una capa de abstracción para operaciones relacionadas con bases de datos, empleando para ello el paquete “sqlite” para interactuar con SQLite, la clase gestiona una base de datos local que almacena información sobre planes, utilizando un modelo *Plan*. Incluye un método privado *\_initDB* que sirve para inicializar y crear la base de datos con una tabla *Plans* al primer lanzamiento de la aplicación si es que no existiera la base de datos. El método *database* asegura una única instancia de la base de datos mediante un patrón singleton, evitando múltiples inicializaciones.

Esta clase también proporciona operaciones CRUD para trabajar con los datos de la tabla *Plans*, incluyendo *getAllPlans* para recuperar todos los registros, *insertPlan* para agregar un nuevo plan y *deletePlan* para eliminar un plan específico por su *id*.

### 5.4.4 Creación del servicio

Ahora se debe crear el servicio, para ello en el código se deberá definir *PlanService*, esta debe ser una clase que actúe como intermediaria entre la lógica de negocio de la aplicación y la capa de acceso a datos para manejar objetos *Plan*. El siguiente código corresponde a la clase *PlanService*.

```
services/plan_service.dart

import '../repositories/todo_repository.dart';
import '../models/plan.dart';
class PlanService {
  final TodoRepository _todoRepository;
  PlanService(this._todoRepository);
  Future<List<Plan>> getPlans() async {
    return await _todoRepository.getAllPlans();
  }
  Future<int> addPlan(Plan plan) async {
    return await _todoRepository.insertPlan(plan);
  }
  Future<void> deletePlan(Plan plan) async {
    await _todoRepository.deletePlan(plan);
  }
}
```

Haciendo la revisión del código, es posible evidenciar que esta clase utiliza una instancia de *TodoRepository* para realizar operaciones específicas relacionadas con los planes, como obtener todos los planes existentes (*getPlans*), añadir un nuevo plan (*addPlan*) y eliminar un plan existente (*deletePlan*). El servicio encapsula la complejidad de interactuar directamente con la base de datos y proporciona una interfaz sencilla y clara para el resto de la aplicación, siguiendo el principio de separación de preocupaciones.

### 5.4.5 Cambios en el controlador para emplear el servicio

Con la adopción de memoria persistente, se requieren modificaciones en el controlador, estos cambios son esenciales para gestionar adecuadamente el almacenamiento y recuperación de datos de manera eficiente, asegurando que la aplicación pueda interactuar correctamente con la base de datos persistente.

```
controllers/plan_controller.dart

import '../services/plan_service.dart';
import '../models/plan.dart';
import '../utils/list_utils.dart';
class PlanController {
  final PlanService _planService;
  PlanController(this._planService);
```

```
final _plans = <Plan>[];
List<Plan> get plans => List.unmodifiable(_plans);
Future<void> loadPlans() async {
  try {
    _plans.clear();
    final loadedPlans = await _planService.getPlans();
    if (loadedPlans.isNotEmpty) _plans.addAll(loadedPlans);
  } catch (e) {
    throw Exception("Error al cargar planes: $e");
  }
}
Future<void> addPlan(String name) async {
  if (name.isEmpty) name = "Plan";
  name = ListUtils.checkForDuplicates(plans.map((plan) => plan.name), name);
  try {
    int id = await _planService.addPlan(Plan(name: name));
    _plans.add(Plan(id: id, name: name));
  } catch (e) {
    throw Exception("Error al añadir el plan: $e");
  }
}
Future<void> deletePlan(Plan plan) async {
  try {
    await _planService.deletePlan(plan);
  } catch (e) {
    throw Exception('Error al eliminar el plan: $e');
  } finally {
    _plans.removeWhere((item) => item.id == plan.id);
  }
}
}
```

El código expuesto define *PlanController*, una clase destinada a actuar como el controlador en una arquitectura de n capas. Esta clase se encarga de la lógica de negocio relacionada con la gestión de objetos *Plan*, actuando como intermediario entre la vista (UI) y el servicio (*PlanService*) que a su vez interactúa con la capa de datos. *PlanController* mantiene una lista privada de objetos *Plan* que se expone a través de un *getter* como una lista inmodificable para evitar alteraciones externas directas. Proporciona métodos para cargar todos los planes existentes desde el servicio (*LoadPlans*), añadir un nuevo plan asegurándose de que el nombre sea único y no esté vacío (*addPlan*) y eliminar un plan tanto en la capa de servicio como en su lista local (*deletePlan*). El manejo de errores dentro de cada método asegura que se lance una excepción con un mensaje específico si ocurre algún error durante la operación, permitiendo un manejo de errores más granular y facilitando la depuración y el mantenimiento del código.

### 5.4.6 Adaptación del Provider a las nuevas características del controlador

*PlanNotifier* es una clase que extiende *ChangeNotifier*, diseñada para funcionar dentro del patrón de diseño Provider para gestionar el estado relacionado con objetos *Plan*.

A continuación, se expone el código correspondiente a *PlanNotifier*.

providers/plan\_notifier.dart

```
import 'package:flutter/material.dart';
import '../controllers/plan_controller.dart';
import '../models/plan.dart';
class PlanNotifier extends ChangeNotifier {
  final PlanController _planController; bool _isLoading = false;
  String? _errorMessage;
  PlanNotifier(this._planController) {
    _loadPlans();
  }
  bool get isLoading => _isLoading;
  String? get errorMessage => _errorMessage;
  List<Plan> get plans => _planController.plans;
  int get plansCount => plans.length;
  Future<void> _loadPlans() async {
    try {
      _isLoading = true; _errorMessage = null;
      notifyListeners();
      await _planController.loadPlans();
    } catch (e) {
      _errorMessage = "No se pudieron cargar los planes: ${e.toString()}";
    } finally {
      _isLoading = false; notifyListeners();
    }
  }
  Future<void> addPlan(String name) async {
    try {
      _errorMessage = null; notifyListeners();
      await _planController.addPlan(name);
    } catch (e) {
      _errorMessage = "Error al añadir el plan: ${e.toString()}";
    }
    notifyListeners();
  }
  Future<void> deletePlan(Plan plan) async {
    try {
```

```

        _errorMessage = null; notifyListeners();
        await _planController.deletePlan(plan);
    } catch (e) {
        _errorMessage = "Error al eliminar el plan: $e";
    }
    notifyListeners();
}
}

```

El código presentado funciona como un intermediario entre la UI y la lógica de negocio encapsulada en *PlanController*, gestiona las operaciones como cargar, añadir y eliminar planes, mientras proporciona notificaciones a los widgets de la UI sobre los cambios de estado para que puedan reconstruirse con la información más reciente. Inicializa la carga de planes en su constructor, mantiene estados de carga (*\_isLoading*), manejo de errores (*\_errorMessage*), y expone métodos públicos que invocan operaciones en *\_planController* manejando adecuadamente estos estados y asegurando que la UI se actualice mediante *notifyListeners()* en cada cambio significativo de estado. Esta estructura facilita una separación clara de responsabilidades, permitiendo que los cambios en el estado de la aplicación se manejen de manera eficiente y centralizada, mejorando así la reactividad y la mantenibilidad del código.

### 5.4.7 Modificación en el programa principal

Se requiere realizar las modificaciones pertinentes en el programa principal para integrar las capas desarrolladas previamente, en este contexto, el código correspondiente a *PlansApp* deberá adoptar el patrón Provider con el objetivo de gestionar el estado de la aplicación de manera eficiente y desacoplada. Para lograr esta integración, es necesario envolver la aplicación con un *ChangeNotifierProvider*, lo cual facilitará la inyección de una instancia de *PlanNotifier* en el árbol de widgets, permitiendo así una gestión del estado más estructurada y modular.

main.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import './repositories/todo_repository.dart';
import './services/plan_service.dart';
import './controllers/plan_controller.dart';
import './providers/plan_notifier.dart';
import './views/plan_screen.dart';
void main() => runApp(const PlansApp());
class PlansApp extends StatelessWidget {
    const PlansApp({super.key});
    @override
    Widget build(BuildContext context) {

```



```

return ChangeNotifierProvider(
  create: (_) => PlanNotifier(PlanController(PlanService(TodoRepository()))),
  child: const MaterialApp(
    debugShowCheckedModeBanner: false,
    home: PlanScreen()
  ),
);
}
}

```

Debe notarse que el atributo *create* del *ChangeNotifierProvider* mantiene una instancia de *PlanNotifier*, la cual se genera considerando la jerarquía de las capas de controlador, servicio y repositorio, siguiendo ese orden específico. Esta organización asegura que la instancia creada esté debidamente configurada con las dependencias necesarias para su operación, reflejando una estructura coherente y eficiente en la gestión del estado dentro de la aplicación.

### 5.4.8 Modificación de la vista de planes

A continuación, se expone las modificaciones en la vista planes de manera contextualizado para que el lector pueda ubicar donde hacer los cambios, destacando las modificaciones respecto al Capítulo 4 mediante el sombreado de las nuevas líneas. Para indicar las secciones del código que permanecen sin cambios, se utilizarán puntos suspensivos (...).

```

views/plan_screen.dart

import 'package:flutter/material.dart';
...
class PlanScreen extends StatefulWidget {
  ...
}
class _PlanScreenState extends State<PlanScreen> with ScrollToLastItemMixin {
  ...
  @override
  Widget build(BuildContext context) {
    ...
  }
  void _showAddPlanDialog() {
    ...
    onAdd: (String text) {
      final planNotifier = Provider.of<PlanNotifier>(context, listen: false);
      planNotifier.addPlan(text);
      if (planNotifier.errorMessage != null) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(planNotifier.errorMessage!)));
      }
    }
  }
}

```

```

    }
    scrollToLastItem();
  },
  ...
}
Widget _buildList(PlanNotifier planNotifier) {
  if (planNotifier.isLoading) {
    return const Center(
      child: CircularProgressIndicator(),
    );
  }
  if (planNotifier.plans.isEmpty) {
    return const Center(
      child: Text("No hay planes"),
    );
  }
  return ListView.builder(
    ...
  );
}
void _showDeletePlanDialog(PlanNotifier planNotifier, Plan plan) {
  ...
  onDelete: () {
    planNotifier.deletePlan(plan);
    if (planNotifier.errorMessage != null) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(planNotifier.errorMessage!)));
    }
  }
  ...
}

```

Con esto, se concluye el ejercicio.

En este capítulo, se han explorado diversos aspectos del desarrollo de aplicaciones con Flutter, con especial enfoque en la gestión de bases de datos SQLite a través del paquete sqflite y la estructuración de la arquitectura de una aplicación, la cual comprende modelos, repositorios, servicios, controladores y notificadoros (Provider). Se han detallado las prácticas recomendadas para definir y manejar entidades en la base de datos, ejemplificando con la clase Plan y la ejecución de operaciones CRUD básicas.